Deployment Plan: Badminton Training System

Context

Deploy the full-stack badminton training system (React frontend + Express.js backend + PostgreSQL + Redis + optional RabbitMQ) to the cloud. The user initially considered Vercel, but Vercel only supports serverless functions — it cannot host the backend because Socket.IO requires a persistent long-running process. The recommended approach is **Vercel for the frontend** (static SPA) + **Railway for the backend** (Docker container with managed PostgreSQL and Redis).

Architecture

```
[Vercel CDN]                      [Railway]
badminton.vercel.app              backend.railway.app
  Static React SPA                 Express.js + Socket.IO
       |                                |
       +--- HTTPS REST API ---------->+--- Railway PostgreSQL
       +--- WSS WebSocket ----------->+--- Railway Redis
                                      +--- CloudAMQP (optional)
```

Cost: $0/month on free tiers (sufficient for capstone demo)

Code Changes (4 files)

1. Add `trust proxy` to Express — `badminton-backend/src/app.ts`

Railway terminates SSL at its reverse proxy. Without this, Express sees all requests as HTTP.

```
const app = express();
app.set('trust proxy', 1);  // ADD THIS LINE (after line 14)
```

2. Support comma-separated CORS origins — `badminton-backend/src/app.ts`

Allow both local dev and deployed frontend origins via a single env var:

```
// Replace lines 18-21 with:
const allowedOrigins = (process.env.FRONTEND_URL ||
'http://localhost:3000').split(',');
app.use(cors({
  origin: allowedOrigins.length === 1 ? allowedOrigins[0] :
allowedOrigins,
  credentials: true,
}));
```

3. Match CORS in Socket.IO — `badminton-backend/src/websocket/socket.handler.ts`

Apply the same multi-origin support to Socket.IO (line 9-12):

```
const allowedOrigins = (process.env.FRONTEND_URL ||
'http://localhost:3000').split(',');
this.io = new Server(server, {
  cors: {
    origin: allowedOrigins.length === 1 ? allowedOrigins[0] :
allowedOrigins,
    credentials: true,
  },
});
```

4. Optimize Dockerfile with multi-stage build — `badminton-backend/Dockerfile`

Reduce image size by excluding devDependencies and source files from production:

```
# Build stage
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Production stage
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=builder /app/dist ./dist
EXPOSE 5000
CMD ["node", "dist/server.js"]
```

Deployment Steps

Phase 1: Deploy Backend on Railway

1. **Create Railway account** — sign up at https://railway.app with GitHub
2. **New project** — "Deploy from GitHub repo" → select `capstone` repo
3. **Configure service** — set Root Directory to `badminton-backend`
4. **Add PostgreSQL** — click "New" → "Database" → "PostgreSQL", link to backend service
5. **Add Redis** — click "New" → "Database" → "Redis", link to backend service
6. **Set environment variables** in Railway:

Variable	Value
`NODE_ENV`	`development` (so TypeORM auto-creates tables on first deploy)
`JWT_SECRET`	Generate with `openssl rand -hex 32`
`FRONTEND_URL`	`https://your-app.vercel.app` (update after Phase 2)

> `DATABASE_URL`, `REDIS_URL`, and `PORT` are auto-injected by Railway.

7. **Deploy** — push to GitHub, verify with `curl https://<railway-url>/health`
8. **Run indexes** — connect to Railway PostgreSQL and run `scripts/add_performance_indexes.sql`

Phase 2: Deploy Frontend on Vercel

1. **Create Vercel account** — sign up at https://vercel.com with GitHub
2. **Import project** — select `capstone` repo, set Root Directory to `badminton-frontend`
3. **Framework preset** — Vercel auto-detects Create React App
4. **Set environment variables** (these are baked at build time):

Variable	Value
`REACT_APP_API_URL`	`https://<railway-url>/api`
`REACT_APP_SOCKET_URL`	`https://<railway-url>`

5. **Deploy** — Vercel builds and hosts the SPA with automatic HTTPS + SPA routing
6. **Update Railway** — set `FRONTEND_URL` to the Vercel URL (triggers backend redeploy)

Phase 3: Deploy RabbitMQ via CloudAMQP

1. **Create CloudAMQP account** — sign up at https://www.cloudamqp.com/
2. **Create instance** — select free "Little Lemur" plan (1M messages/month)
3. **Copy AMQP URL** — from the instance dashboard (format: `amqps://user:pass@host/vhost`)
4. **Add to Railway** — set `RABBITMQ_URL` env var in the backend service with the CloudAMQP URL
5. **Verify** — Railway redeploys automatically; check logs for ✅ `RabbitMQ connected and configured`

> Note: CloudAMQP uses `amqps://` (TLS) instead of `amqp://`. The `amqplib` library handles both transparently.

Verification Checklist

1. `curl https://<railway-url>/health` → `{"status":"ok",...}`
2. Open `https://<vercel-url>` → login page renders, no CORS errors in console
3. Register a new account → redirects to training page
4. Refresh page → session persists (JWT + Redis working)
5. Open DevTools Network tab → WebSocket connection upgrades successfully
6. Navigate directly to `/performance` → page loads (SPA routing works)

7. Create an athlete → appears in athlete list
8. Start/stop a training session → session saved (without RabbitMQ, no shots will arrive, but session lifecycle works)

Files Modified

File	Change
`badminton-backend/src/app.ts`	Add trust proxy, multi-origin CORS
`badminton-backend/src/websocket/socket.handler.ts`	Multi-origin CORS for Socket.IO
`badminton-backend/Dockerfile`	Multi-stage build
`badminton-frontend/`	No code changes (env vars set at deploy time)